

**Supplementary Material for “A
Centralized System Performance
Monitoring Infrastructure”**

Table of Contents

List of Figures	iii
List of Tables	iv
1 Specification: EVU-APMU Interface	1
2 Specification: Advanced Performance Monitoring Unit	4
2.1 APMU Control and Status Registers	5
2.2 APMU Ports	6
2.3 Event Filter	7
2.4 Counter Block	7
2.5 APMU Core and Memory Infrastructure	14
2.6 APMU Interconnect	15
2.7 Software Design Considerations	15
3 Specification: CVA6-EVU	17
3.1 EVU-APMU Interface	18
3.1.1 Physical Layer	18
3.1.2 Logical Layer	18
3.2 CVA6-EVU Event Table	19
3.3 Configuration Interface	19
References	23

List of Figures

1.1	The fields of an event packet.	1
1.2	Event Info bits transmitting request latency and transaction size.	2
2.1	APMU Architecture.	5
2.2	Timer register of the APMU core.	6
2.3	BootAddr register of the APMU core.	6
2.4	Status register of the APMU core.	6
2.5	APMU Counter Block.	9
2.6	Format of a XLEN-bit APMU Counter.	9
2.7	Format of EventSelCfg register.	10
2.8	Format of EventInfoCfg register.	10
3.1	Event ID field bit assignment for the CVA6-EVU (default: N_EVENTS = 4).	18
3.2	Event Info field bit assignment for the CVA6-EVU (default: N_MARKER = 4).	19

List of Tables

1.1	An exemplified EVU Event Table.	3
2.1	Counter Mode of Operation.	8
2.2	ALU operations and their opcodes.	13
2.3	Example of an APMU Memory Map.	16
3.1	CVA6-EVU configuration register memory map (default: <code>N_EVENTS = 4</code> , <code>N_MARKERS = 4</code> , <code>core_stride = 48</code> bytes).	21
3.2	CVA6-EVU Event Table. Event IDs 1–22 map directly to the CVA6 hard- ware performance counter event selector encoding. Event ID 0x17 is a cus- tom event used for PC marker.	22

Chapter 1

Specification: EVU-APMU Interface

This specification discusses the interface that connects the output port of an Event Unit IP (EVU) to an input port on the Advanced Performance Monitoring Unit IP (APMU). An EVU is a hardware monitoring IP that can be designed and installed in any desired platform component to monitor hardware events and collect additional information related to them. The component in which the EVU is embedded for monitoring is called a *monitored* component. The EVU sends the observed event information to the APMU for counting and other purposes.

The interface is composed of two layers: the logical layer, which describes the information that the EVU sends to the APMU for each observed event, and the physical layer, which describes the physical connection used to transmit this information. These layers are detailed below:

1. Logical Layer

The EVU sends an event packet for each individual event that it observes in the monitored component. Each event packet is composed of three fields: **Event ID**, **Event Info**, and **Source ID**, as shown in Figure 1.1. They are explained below:

Source ID	Event Info	Event ID
-----------	------------	----------

Figure 1.1: The fields of an event packet.

(a) **Event ID** (Mandatory)

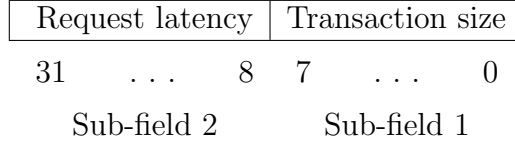


Figure 1.2: Event Info bits transmitting request latency and transaction size.

The **Event ID** field encodes the type of event that was observed by the EVU in its monitored component. The **Event ID** is encoded as an unsigned integer. All events of an EVU must be mapped to unique **Event IDs**.

(b) **Event Info** (Optional)

An EVU can also transmit additional information alongside each event using the **Event Info** field. This field can be split into sub-fields where each sub-field represents a specific type of event information.

For example, in the case of an EVU monitoring read or write requests through a bus, transmitting information about the response latency and the transaction size might be beneficial. Therefore, the number of sub-fields for the **Event Info** field for these events would be 2, as shown in Figure 1.2.

(c) **Source ID** (Optional)

The *source* of an event is the component external to the monitored component whose behaviour led to the generation of said event. For example, the processor that initiates a cache access is the source of that access and all events that are generated due to that access. Each potential event source is represented by a unique **Source ID**. Similar to **Event ID**, this field is also encoded as an unsigned integer.

Each EVU-APMU interface must have a document that helps users program the APMU to filter the event packets being transmitted through it. This document is called the *event table*. The event table contains the list of events the EVU can transmit through that interface, along with its **Event ID** encoding, the **Event Info** bits associated with each event, and the **Source ID** encoding. The example of a reduced event table is given in Figure 1.1. This is event table of an AXI-based EVU. The **Source ID** encoding is omitted in this example.

Lastly, the **Event ID** and **Source ID** encodings are not expected to be consistent across EVUs types. Therefore, different types of EVUs can re-use the same IDs to represent events/source.

Event	Event ID	Event Info
No event	0	NA
Read request	1	Size of transaction, unaligned transfer
Write request	2	
Read response	3	Request latency, clock spent in contention
Write response	4	

Table 1.1: An exemplified EVU Event Table.

2. Physical Layer

The physical layer defines the physical connection that is used to connect the EVU to a port on the APMU. The EVU can choose one of the provided physical layers for its interface, or it can choose to implement a different one; but in that case, they must provide a specification for the interface.

(a) **Parallel Interface**

The parallel interface is the simplest interface, wherein the EVU sends one event packet per clock cycle using a set of parallel wires. The size of the interface is equal to the size of the event packet of the EVU. This interface can only send one event packet per clock cycle but each packet can have additional event-related information packed in it, along side the **Event ID**.

(b) **One-hot Interface**

In this interface, the EVU sends a set of parallel wires to the APMU port, where each bit of the wire represents an individual event. This interface allows the EVU to send multiple event packets per clock cycle, where each packet is composed of only one bit.

If the EVU and APMU are running in different clock domains then the interface must have clock domain crossing FIFOs (CDC FIFOs) to help facilitate asynchronous communication between the IPs.

Chapter 2

Specification: Advanced Performance Monitoring Unit

This specification presents the Advanced Performance Monitoring Unit (APMU) that enables users to implement various event-dependent software mechanisms. The APMU, as shown in Figure 2.1, has the following features:

1. Each EVU output port connects to a dedicated port on the APMU, represented by a unique `Port ID`.
2. A configurable number of counter blocks, wherein each counter block has one counter, two configuration registers (`EventSelCfg` and `EventInfoCfg`) and an arithmetic-logic unit (ALU). The counter blocks have support to generate interrupts in case of overflow.
3. Each counter block has its separate event filter which processes the event packets received from all APMU ports to generate the corresponding signals to update its counter.
4. An instruction processor, with complementary instruction and data scratchpad memories (SPMs), capable of executing program code stored in said SPMs.
5. A subordinate port that allows components, external to the APMU, to program the counter blocks, the processor and its SPMs.
6. A manager port that connects to the rest of the platform, allowing the APMU core to read/write to non-APMU mapped memory addresses.

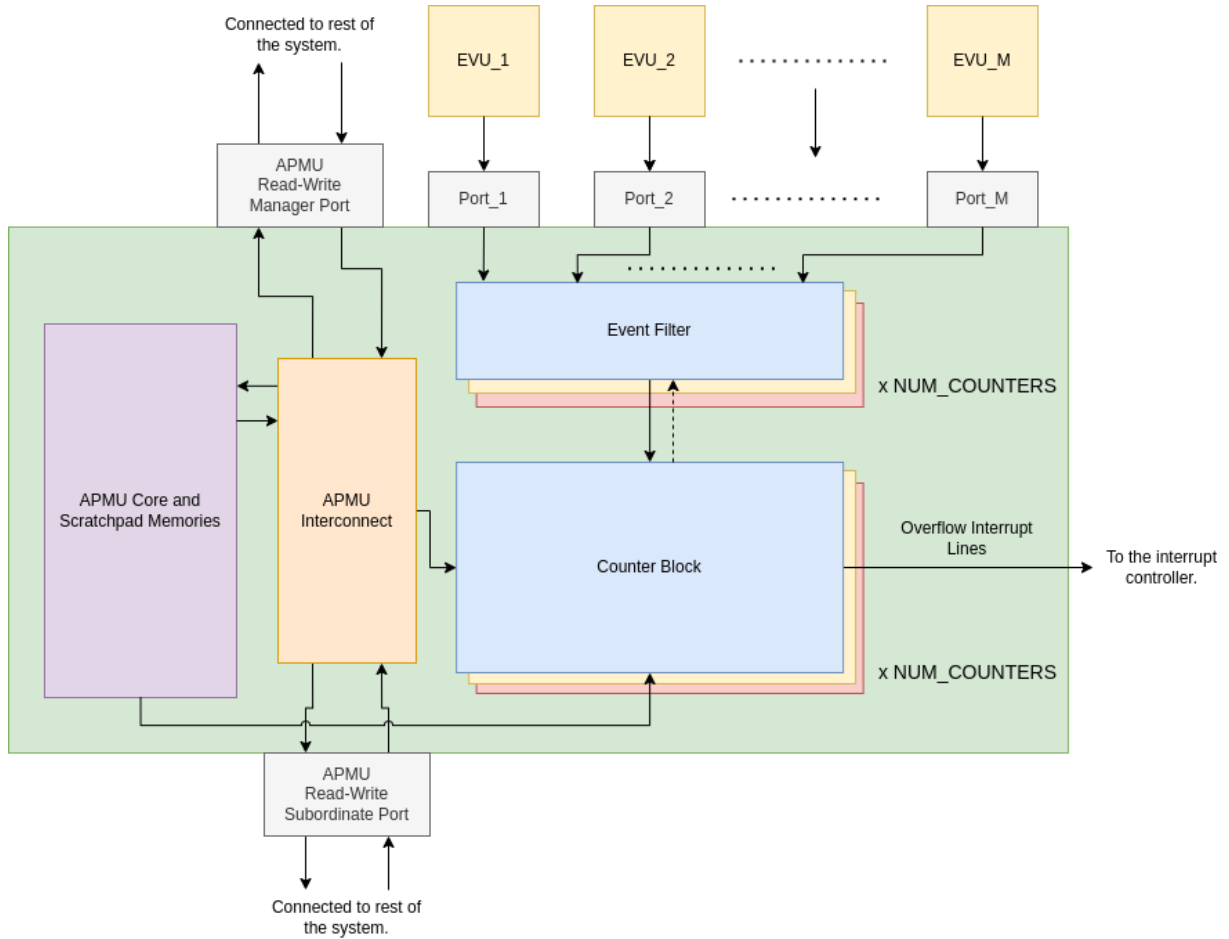


Figure 2.1: APMU Architecture.

2.1 APMU Control and Status Registers

The core has three control and status registers:

1. APMU Timer

The APMU has a 64-bit cycle counter. It is a read-only register necessary for time-keeping operations.

2. Boot Address Register



Figure 2.2: **Timer** register of the APMU core.

The APMU has a boot address register (**BootAddr**), which is read- and write-able. The core always boots from the address written into this register. The bit width of the register (**YLEN**) is dependent on the bit width of the APMU processor.

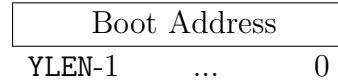


Figure 2.3: **BootAddr** register of the APMU core.

3. Status Register

The status register (**Status**) is a 32-bit read- and write-able register that is required to reset the APMU core. In the current specification, the 0th bit of the register is called the **Stall** bit. If this bit is set, the APMU core stalls. The core will restart from the address written into **BootAddr** register, described in Point 2 of 2.1.

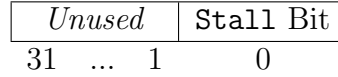


Figure 2.4: **Status** register of the APMU core.

2.2 APMU Ports

The APMU has multiple ports, one for each EVU interface connected to it. An EVU can be connected to multiple APMU ports through multiple interfaces. The output of every APMU port is passed downstream to a set of counters through their respective event filter. For each port, the size of this counter set can range from one to all the counters in the APMU.

Each APMU port is represented by a unique **Port ID**. Since EVUs of different types can re-use the same **Event ID** for different events, it is important to add another layer of

identification to the design. This is achieved through the **Port ID**, as the **Port ID** helps identify the EVU connected to it. Once the EVU is known, the user can also decode the events received by comparing its **Event ID** to the event table of the interface.

2.3 Event Filter

Each counter block, discussed in Section 2.4, has its own event filter. This module processes all the event packets received from the APMU ports and depending on the configuration registers of its counter, filters out unnecessary events. The configuration registers specify a set of events, that its counter should either count or operate upon, using the **Event ID**, **Source ID** and **Port ID** fields. These events are called *selected events*. The event filters should be capable of accepting multiple selected events received simultaneously from one or more ports. As mentioned in Section 2.2, each APMU port is fanned out to either all or a subset of event filters in the APMU. The ports can send the event packets received from their respective EVU downstream to only those the event filters, and by extension counters, that are connected to them.

Before continuing this section, please read Table 2.1.

2.4 Counter Block

Each APMU counter is bundled together with its configuration registers and a programmable ALU to constitute a *counter block*, as shown in Figure 2.5. The specification does not limit the number of counter blocks in the APMU. This section is divided into four parts, explaining each component of the counter block.

1. APMU Counter

The APMU counter has a parameterizable bit width **XLEN**, as shown in Figure 2.6. The two most significant bits of the counter are called its pending and overflow bits, while the rest are used for standard counter operation. The pending bit is set whenever the counter is incremented during its count or functional operation mode. The overflow bit is set when the **XLEN-2** bit-wide counter overflows. If overflow interrupt is enabled in the **EventInfoCfg** register then the counter triggers an interrupt upon overflow.

Counter Mode of Operation

An APMU counter can be programmed to operate in two modes: *count* or *functional* mode. The operation mode of the counter is controlled via the **Operation** Bit in the **EventInfoCfg** register, discussed in Point 2b of Section 2.4. The two modes are explained below:

1. Count mode

When the **Operation** bit is reset to 0, the counter is said to be operating in count mode. In this mode, the counter is incremented by the number of selected events received across all APMU ports connected to its event filter.

2. Functional mode

When the **Operation** bit is set to 1, the counter is said to be operating in functional mode. In this mode, the event info bits associated with the selected event packet are operated upon by the ALU of the counter. The output computed by the ALU is then updated into the counter. This mode should only be set in scenarios where the counter is not expected to receive multiple selected events in one clock cycle.

If the event filter does receive multiple selected events while its counter is in functional mode then the filter can arbitrarily pick the event info bits of any one of the selected event packets. This is because otherwise the ALU hardware will become unnecessarily complicated if it is expected to operate on event info bits from multiple event packets simultaneously.

Table 2.1: Counter Mode of Operation.

2. Configuration registers

The counter block has two configuration registers: the Event Selection Register (**EventSelCfg**), which is used to specify the IDs of the selected event, and the Event Info Register (**EventInfoCfg**), which is used to specify details regarding the functional operation mode of the counter. The formats of the registers are detailed below:

(a) Event Selection Register (**EventSelCfg**)

The **EventSelCfg** register has the fields described in Figure 2.7. Let **ESLEN** be the bit width of the **EventSelCfg** register. The **Event ID Value** and **Event ID Mask** are used to specify the set of selected events that the counter should count or operate on. The event filter will only accept those events whose **Event IDs**

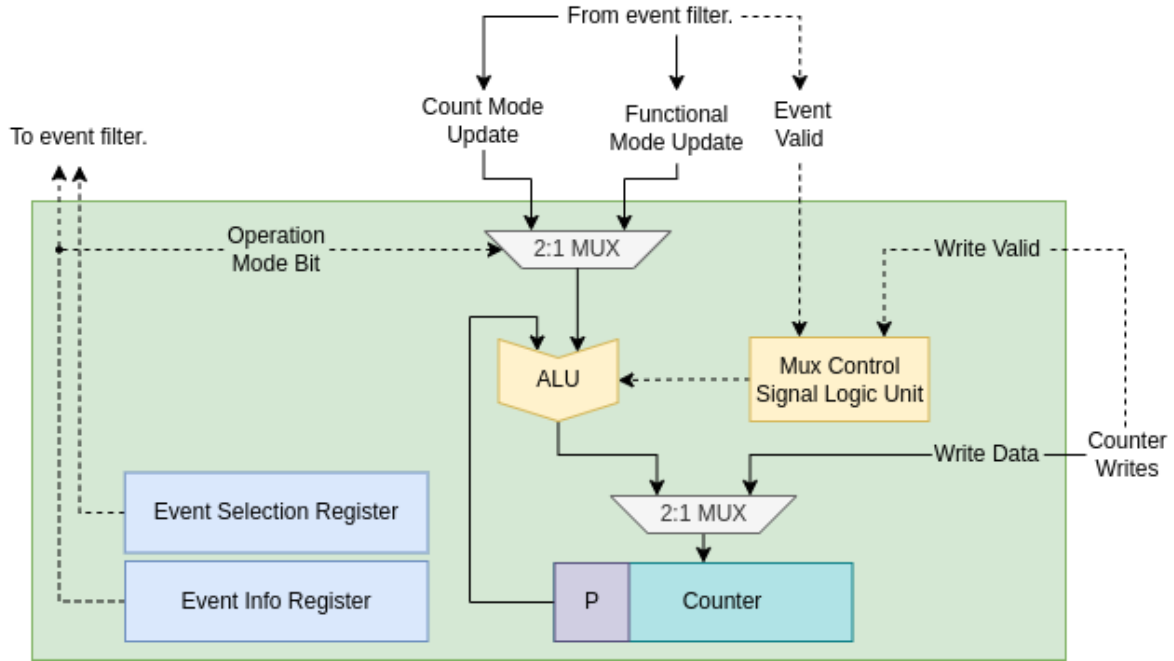


Figure 2.5: APMU Counter Block.

Pending Bit	Overflow Bit	[XLEN-2]-bit Counter			
XLEN-1	XLEN-2	XLEN-3	...	0	

Figure 2.6: Format of a XLEN-bit APMU Counter.

satisfy the following boolean expression:

$$(\text{Event ID AND Event ID Mask}) == \text{Event ID Value}$$

The size of these fields is equal to the minimum number of bits needed to specify the **Event ID** of any of the events that can be sent by the EVUs connected to this particular counter through the APMU ports.

Similarly, the event filter can be configured to filter event packets based on their **Source ID** and the **Port ID** of the port that received the packet. Counters will only increment for events whose **Event ID**, **Source ID**, and receiving port's **Port ID** are in the set specified by the **EventSelCfg** register. The size of the **Source ID Mask** and **Source ID Value** is computed similar to the **Event ID Mask** and **Event ID Value**. On the other hand, the size of the

Port ID Value	Port ID Mask	Source ID Value	Source ID Mask	...	
ESLEN-1					
			...	Event ID Value	Event ID Mask

0

Figure 2.7: Format of EventSelCfg register.

Port ID Mask and **Port ID Value** is equal to the minimum number of bits needed to store the **Port ID** of any of the APMU ports connected to the counter.

Note: There are no restrictions on the number of bits used for any of the IDs. For instance, consider an APMU connected to three EVUs. It will only have three ports. In this case, the system architect can use the following **Port ID**s: 3'b001, 3'b010, 3'b100. This allows them to configure counters to count events transmitted by different combinations of ports. If **Port ID Mask** and **Port ID Value** are set to 3'b001 and 3'b000 then any event packet arriving at a port with ID 3'bXX0 will be accepted; here X represent "do not care" bits. This means both ports 3'b100 and 3'b010 will be considered but not 3'b001.

(b) **Event Info Register (EventInfoCfg)**

Overflow Intr Enable	Operation Mode	Value Upper	Value Lower	...
EILEN-1				
...	ALU Opcode	EISF End	EISF Start	0

Figure 2.8: Format of EventInfoCfg register.

The **EventInfoCfg** register has the fields described in Figure 2.8. Let **EILEN** be the bit width of the **EventInfoCfg** register. The bit width of some of the fields is dependent on the types of EVUs connected to the APMU. The explanation of each field is as follows:

i. **EISF Start** and **EISF End**

Event Info Sub-field Start (EISF Start) and End (EISF End) fields are used to specify the starting and ending bits of the **Event Info** field that the ALU should operate upon. This is because these bits can be subdivided to pack various metadata together, as discussed in Point 1b of Appendix 1.

Therefore, the user needs to specify which of the metadata sub-field, i.e., which contiguous set of the event info bits, the ALU should operate upon. Both of these fields can take any value between 0 and the maximum number of bits used for the **Event Info** field out of all the connected EVUs. The ALU slices the **Event Info** field associated with a selected event according to these starting and ending bits, both inclusive, as written in **EISF Start** and **EISF End**, respectively. The ALU uses these *sliced event info bits* as an input operand, while the other input operand is the counter data.

ii. **ALU Opcode**

The ALU of the counter block can be programmed to perform specific operations by writing to this field. A set of supported opcode and their corresponding operations are given in Table 2.2. The supported ALU operations are described below:

A. **Addition**

The ALU adds the sliced event info bits to the APMU counter.

B. **KeepMax**

The ALU compares the sliced event info bits against the counter value and stores the larger of the two back into the counter. Using this operation, the ALU can be programmed to capture the maximum encountered value of an event metadata.

C. **KeepMin**

The ALU compares the sliced event info bits against the counter value and stores the smaller of the two value back into the counter. Using this operation, the ALU can be programmed to capture the minimum encountered value of an event metadata.

D. **Increment: Eq**

The ALU compares the value of the sliced event info bits to the value written in the **Value_L** field of **EventInfoCfg**. If the two values are equal then the counter is incremented by 1.

E. **Increment: NotEq**

The ALU compares the value of the sliced event info bits to the value written in the **Value_L** field of **EventInfoCfg**. If the two values are not equal then the counter is incremented by 1.

F. **Increment: LessThan**

The ALU compares if the value of the sliced event info bits is less than the value written in the **Value_L** field of **EventInfoCfg**. If yes then the counter is incremented by 1.

G. **Increment: GreaterThan**

The ALU compares if the value of the sliced event info bits is greater than the value written in the **Value_L** field of **EventInfoCfg**. If yes then the counter is incremented by 1.

H. **Increment: LessThanEqual**

The ALU compares if the value of the sliced event info bits is less than or equal to the value written in the **Value_L** field of **EventInfoCfg**. If yes then the counter is incremented by 1.

I. **Increment: GreaterThanEqual**

The ALU compares if the value of the sliced event info bits is greater than or equal to the value written in the **Value_L** field of **EventInfoCfg**. If yes then the counter is incremented by 1.

J. **Increment: InRange**

The ALU compares if the value of the sliced event info bits is in the range $[\text{Value_L}, \text{Value_U}]$ (both inclusive). If yes then the counter is incremented by 1.

K. **Increment: NotInRange**

The ALU compares if the value of the sliced event info bits is in the range $[\text{Value_L}, \text{Value_U}]$ (both inclusive). If not then the counter is incremented by 1.

L. **Add: X**

These operations are similar to the **Increment: X** operations. The only difference is that if the evaluation condition is true then the sliced event info bits are added to the counter. Here, **X** can be any of the following relational operator: **Equal**, **NotEqual**, **LessThan**, **GreaterThan**, **LessThanEqual**, **GreaterThanEqual**, **InRange**, **NotInRange**.

iii. **Value Lower (Value_L) and Value Upper (Value_U)**

The Value Lower and Value Upper fields are used in conjunction with the

Operation	Opcode
Addition	00000
KeepMax	00001
KeepMin	00010
Increment: Eq	00011
Increment: NotEq	00100
Increment: LessThan	00101
Increment: GreaterThan	00110
Increment: LessThanEqual	00111
Increment: GreaterThanEqual	01000
Increment: InRange	01001
Increment: NotInRange	01010
Add: Eq	01011
Add: NotEq	01100
Add: LessThan	01101
Add: GreaterThan	01110
Add: LessThanEqual	01111
Add: GreaterThanEqual	10000
Add: InRange	10001
Add: NotInRange	10010

Table 2.2: ALU operations and their opcodes.

ALU to evaluate the specified boolean condition. The values written into these two fields are treated as **Unsigned Integer**. The specification does not limit the size of the two fields.

iv. **Operation Mode**

This bit is used to specify the operation mode of the APMU counter, as discussed in Table 2.1. If this bit is reset to 0 then the counter will only track the number of selected events and the ALU will not perform any event info-specific operation. If this bit is set to 1 then the counter is in functional mode and will update based on the ALU computation result.

v. **Counter Overflow Enable bit**

The MSB of the **EventInfoCfg** register is the **Counter Overflow Enable** bit. If the bit is set then upon overflow the counter will generate an overflow

interrupt. Otherwise, it will not generate any interrupt. In both cases, the counter resumes operation after an overflow without resetting its overflow bit. This must be handled in software.

2.5 APMU Core and Memory Infrastructure

The APMU has an instruction processor capable of executing program code. It should be supported by instruction and data scratchpad memories (SPMs). The specification also endorses additional functionalities for the APMU core to improve its performance when executing event-dependent programs, such as an exclusive datapath to access counters and to extend its capabilities, such as the ability to write to other memory locations in the system. These functionalities are mentioned below:

1. **Support to read and write counters efficiently.**

The core should have an efficient datapath to reduce the access latencies for the APMU counters.

2. **Support to read and write to rest of the platform memory.**

The core should be able to read and write from non-APMU mapped memory addresses. This allows the core to interact with other system components and can be useful for implementing event-dependent mechanisms.

3. **Support for Wait-for-X functionality.**

The counters have two special bits: pending and overflow, as detailed in Section 2.4. The pending is set to 1 whenever the counter increments in count or functional mode. The overflow bit is set to 1 when the XLEN-2 bit counter overflows. To effectively use these bits, the specification proposes two functionalities:

- (a) **Wait-for-Pending**

Under this functionality, the core enters a busy wait mode, polling the pending bit of a subset of counters. It exits the busy wait mode when the pending bit of any of the specified counters is set to 1. When the core exits the Wait-for-Pending, it also resets the pending bit of the counters that were set.

- (b) **Wait-for-Overflow**

Similarly, under this functionality, the core enters a busy wait mode, polling the overflow bit of a subset of counters. It exits the busy wait mode when the

overflow bit of any of the specified counters is set to 1. When the core exits the Wait-for-Overflow, it also resets the overflow bit of the counters that were set.

4. Support to trigger interrupts.

The core should be able to trigger interrupts in other application cores and devices.

5. External support to reset and halt the core.

The APMU should have support to halt and reset the APMU core using external platform components. This feature is aided by the **Status** and **BootAddr** registers, discussed in [2.1](#).

2.6 APMU Interconnect

The APMU also has an internal interconnect that enables communication between the APMU core and other APMU components such as the SPMs, counters, configuration registers, etc. The APMU has a manager port that allows the APMU core to access non-APMU memory addresses in the platform. Additionally, the APMU also has a subordinate port that allows other components in the platform to access internal APMU components. This allows external platform components to initialize and configure the APMU for operation.

2.7 Software Design Considerations

To facilitate integration in virtualized systems, the APMU counter blocks are spaced apart in the physical memory. This allows the hypervisor to place different counter blocks on different physical pages. Each of these pages can then be mapped to the virtual memory of a different virtual machine. An example memory map is shown in [Table 2.3](#) where each counter block is spaced exactly one page apart.

Counter Block	Register	Address
NA	APMU Timer	base
	APMU Core Status Register	base + 0x8
	APMU Core Boot Address	base + 0xc
0	Event Selection Register	base + 0x10
	Event Info Register	base + 0x14
1	Event Selection Register	base + 0x18
	Event Info Register	base + 0x1c
...	...	...
31	Event Selection Register	base + 0x108
	Event Info Register	base + 0x10c
0	Counter	base + page_size
1	Counter	base + 2 × page_size
...	...	...
31	Counter	base + 32 × page_size

Table 2.3: Example of an APMU Memory Map.

Chapter 3

Specification: CVA6-EVU

This specification describes the CVA6 Performance Counters Event Unit (CVA6-EVU), an EVU designed to be instantiated inside the CVA6 RISC-V core's hardware performance counter (HPC) subsystem. Each CVA6 core in the system is equipped with one dedicated instance of this EVU. The CVA6-EVU monitors a configurable set of micro-architectural events sourced from the CVA6 hardware performance counters and, additionally, tracks program control-flow progress via Program Counter (PC) markers. It transmits the observed event information to the APMU using the EVU-APMU interface described in Chapter 1.

The CVA6-EVU is composed of two internal logic blocks, each serving a distinct function:

1. Event ID Matching and Routing

This block continuously monitors the status signals of the CVA6 hardware performance counters. It supports up to `N_EVENTS` (default: 4) simultaneously monitored events, each selected by a configurable **Event ID** drawn from the CVA6 event set (see Section 3.2). A multi-channel MUX-based selector checks up to `N_EVENTS` programmable event IDs in parallel. When a monitored event is asserted in a given clock cycle, the corresponding status bit in the EVU response vector is driven high.

2. PC Marker

This block interfaces directly with the `commit_pc` and `commit_ack` signals from the commit stage of the CVA6 six-stage in-order pipeline. It compares each committed

instruction's program counter against a set of up to `N_MARKERS` (default: 4) configurable 64-bit PC marker addresses. When a committed instruction's PC matches a configured marker address while `commit_ack` is asserted, the corresponding marker-hit status bit is driven high for that clock cycle.

3.1 EVU-APMU Interface

3.1.1 Physical Layer

The CVA6-EVU uses the **one-hot interface** as defined in Chapter 1. Each bit of the interface wire corresponds to a single event. This allows the EVU to report multiple concurrent events in a single clock cycle.

3.1.2 Logical Layer

Each clock cycle, the CVA6-EVU assembles a single event packet and transmits it to the APMU port assigned to that core. The packet fields are packed as follows:

1. Event ID

The **Event ID** field is `N_EVENTS` bits wide. Each bit corresponds to one of the up to `N_EVENTS` programmable performance counter event slots. Bit i is set to 1 in a given clock cycle if the CVA6 hardware performance counter event assigned to slot i is asserted in that cycle.

Event Slot 3	Event Slot 2	Event Slot 1	Event Slot 0
Bit 3	Bit 2	Bit 1	Bit 0

Figure 3.1: Event ID field bit assignment for the CVA6-EVU (default: `N_EVENTS` = 4).

Note: When the PC Marker event ID (0x17) is assigned to one of the event slots (say slot k), bit k of the **Event ID** field acts as a *marker-hit indicator*. It is set to 1 in cycles where any PC marker match is detected, indicating to the APMU that the **Event Info** field carries valid marker hit data for that cycle.

2. Event Info

The **Event Info** field is **N_MARKER** bits wide. Each bit corresponds to one of the up to **N_MARKER** configured PC marker addresses. Bit j of the **Event Info** field is set to 1 in cycles where the committed instruction's PC matches marker address j .

Marker 3 Hit	Marker 2 Hit	Marker 1 Hit	Marker 0 Hit
Bit 3	Bit 2	Bit 1	Bit 0

Figure 3.2: Event Info field bit assignment for the CVA6-EVU (default: **N_MARKER** = 4).

3. Source ID

The **Source ID** field is not used in the current implementation and is always driven to zero.

3.2 CVA6-EVU Event Table

The CVA6-EVU event table is given in Table 3.2. Events 1–22 are sourced from the CVA6 hardware performance counter event set [1, 2]. Event ID 0x17 (decimal 23) is a custom event reserved for PC marker tracking; it is assigned to a performance event slot via the AXI-Lite configuration registers (Section 3.3).

When a performance event slot is configured with Event ID 0x17, the slot's corresponding bit in the **Event ID** field is asserted in any cycle where at least one PC marker match occurs.

3.3 Configuration Interface

The CVA6-EVU is dynamically configured at runtime via a set of memory-mapped registers. Each CVA6 core is allocated **N_EVENTS** + **N_MARKERS** configuration registers, subdivided as follows:

1. Event ID Registers (**EventID0**–**EventIDk**)

N_EVENTS registers, one per performance event slot. Only the 5 least significant bits of each register are meaningful, as the CVA6 event ID space requires at most 5

bits. Writing a value to `EventID $k$` configures the CVA6-EVU to monitor the CVA6 hardware performance counter event with that ID in slot k . Setting `EventID $k$` = 0x17 configures slot k as the PC marker indicator slot.

2. PC Marker Registers (PCTRACK0–PCTRACK j)

`N_MARKERS` registers, one per PC marker slot. Writing a 64-bit virtual address to `PCTRACK $j$` configures the PC Marker Block to assert marker-hit bit j whenever a committed instruction’s PC matches that address.

The per-core memory map is shown in Table 3.1. Let `base` be the base address of the CVA6-EVU configuration space and `core_stride` be the number of bytes allocated per core. Core n ’s configuration space begins at `base` + $n \times \text{core_stride}$. Each Event ID register occupies 4 bytes (word-aligned) and each PC Marker register occupies 8 bytes (doubleword-aligned), giving a `core_stride` of $4 \times \text{N_EVENTS} + 8 \times \text{N_MARKERS}$ bytes.

Upon reset, all `EventID` and `PCTRACK` registers hold the value zero. A zero `EventID` corresponds to “no event” (Event ID 0) and results in no assertion from that slot. A zero `PCTRACK` value means that marker slot is not active. Software must write valid configurations before enabling the APMU counters associated with this EVU.

Core	Register	Address	Width
0	EventID0	base	32-bit
	EventID1	base + 0x4	32-bit
	EventID2	base + 0x8	32-bit
	EventID3	base + 0xC	32-bit
	PCTRACK0	base + 0x10	64-bit
	PCTRACK1	base + 0x18	64-bit
	PCTRACK2	base + 0x20	64-bit
	PCTRACK3	base + 0x28	64-bit
1	EventID0	base + core_stride	32-bit
	EventID1	base + core_stride + 0x4	32-bit
	EventID2	base + core_stride + 0x8	32-bit
	EventID3	base + core_stride + 0xC	32-bit
	PCTRACK0	base + core_stride + 0x10	64-bit
	PCTRACK1	base + core_stride + 0x18	64-bit
	PCTRACK2	base + core_stride + 0x20	64-bit
	PCTRACK3	base + core_stride + 0x28	64-bit
n	EventID0	base + $n \times$ core_stride	32-bit
	...	...	...

Table 3.1: CVA6-EVU configuration register memory map (default: N_EVENTS = 4, N_MARKERS = 4, core_stride = 48 bytes).

Event ID	Event Name	Description
0	No event	No event
1	L1 I-Cache Misses	Number of misses in the L1 instruction cache
2	L1 D-Cache Misses	Number of misses in the L1 data cache
3	ITLB Misses	Number of misses in the instruction TLB
4	DTLB Misses	Number of misses in the data TLB
5	Load Accesses	Number of data memory load operations
6	Store Accesses	Number of data memory store operations
7	Exceptions	Valid exceptions encountered
8	Exception Handler Returns	Returns from an exception handler
9	Branch Instructions	Number of branch instructions encountered
10	Branch Mispredicts	Number of branch mispredictions
11	Branch Exceptions	Number of valid branch exceptions
12	Call	Number of call instructions
13	Return	Number of return instructions
14	MSB Full	Scoreboard is full
15	Instruction Fetch Empty	Number of invalid instructions in the instruction fetch
16	L1 I-Cache Accesses	Number of accesses to the instruction cache
17	L1 D-Cache Accesses	Number of accesses to the data cache
18	L1 Cache Line Eviction	Number of data cache line evictions
19	ITLB Flush	Number of instruction TLB flushes
20	Integer Instructions	Number of integer instructions retired
21	Floating Point Instructions	Number of floating-point instructions retired
22	Pipeline Stall	Cycles the pipeline is stalled during operand read
23 (0x17)	PC Marker	Set when any PC marker is matched

Table 3.2: CVA6-EVU Event Table. Event IDs 1–22 map directly to the CVA6 hardware performance counter event selector encoding. Event ID 0x17 is a custom event used for PC marker.

References

- [1] F. Zaruba and L. Benini. The cost of application-class processing: Energy and performance analysis of a linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, 27(11):2629–2640, 2019.
- [2] OpenHW Group. CVA6 RISC-V processor repository. <https://github.com/openhwgroup/cva6>, 2023. Version v4.2.0, accessed 2025.